

Table Operations

Sure! Let's dive into the concept of creating tables in Apache Cassandra, which is a NoSQL database.

In simple terms, a table in Cassandra is like a container where you store your data, similar to how a bookshelf holds books. Before you can create a table, you first need to create a "keyspace," which is like a library that holds different bookshelves (tables). Each table has a specific structure that defines how the data is organized, including what kind of information it can hold.

For example, when you create a table, you specify its name and the types of data it will contain, like usernames or group IDs. You can also define a primary key, which is like a unique identifier for each entry in the table, ensuring that no two entries are the same. This helps Cassandra know where to store and retrieve the data efficiently.

If you have any other concepts from the content that you would like me to explain, feel free to ask!

Logical Entities: Tables and Keyspaces

Table

- Logical entity that organizes data storage at cluster and node level (according to a declared schema)

Keyspace

- Logical entity that contains one or more tables
- Replication and data centers' distribution is defined at keyspace level
- Recommended one keyspace per application

- Remember that in Cassandra, prior to declaring a table, you first need to create a keyspace. A table will always be created in an existing keyspace. Cassandra stores data in tables whose schema defines the storage of the data at the cluster and node level. The generic create table syntax looks like this

CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS] [keyspace_name.]table_name
( column_definition [, ...]
  PRIMARY KEY (column_name [, column_name ...])
  [WITH table_options
    | CLUSTERING ORDER BY (clustering_column_name order)])
```

```
column_name cql_type_definition [STATIC | PRIMARY KEY] [, ...]
```

- Create table with the optional IF NOT EXISTS parameter, followed by the keyspace_name, if it has not been already stated by use keyspace, and then the table_name. This command creates a new table under the specified keyspace. The IF NOT EXISTS keywords may be used in creating a table.
- Attempting to create an existing table returns an error unless the IF NOT EXISTS option is used. If the option is used and the table already exists, then the error message is suppressed, and no table is created. Columns are defined enclosed in parentheses after the table_name, using a comma separated list to define multiple columns.
- In the column definition, you can also specify, optionally, the PRIMARY KEY parameter which is used to indicate that the column is the only primary key for the table. In a case where you have a primary key formed of multiple columns, then you declare it at the end of the table. Let's take an example.

Static CREATE TABLE example

```
USE intro_cassandra;
```

```
CREATE TABLE users(  
    username text PRIMARY KEY,  
    occupation text,  
    age int;
```

```
USE intro_cassandra;
```

```
CREATE TABLE users(  
    username text,  
    occupation text,  
    age int,  
    PRIMARY KEY(username));
```

```
SELECT * FROM users; //there are no entries in our table so the  
SELECT will yield no data;
```

- Let's now explain the create table statement based on what we just learned. Our table is static, meaning the primary key contains only one column partition key, username. In this case, we can signal the primary key in two ways, just by adding it to the column definition. But the same table would be created if we declared the primary key just like this. Once we've created the table, we can use a SELECT statement, but since we have not inserted any data, the SELECT result will be empty.

Dynamic CREATE TABLE example

```
USE intro_cassandra;  
  
CREATE TABLE groups(  
  groupid int,  
  group_name text STATIC  
  username text,  
  age int,  
  PRIMARY KEY ((groupid), username));  
  
SELECT * FROM groups; //no data
```

- Now let's create a dynamic table as well. A dynamic table means that our primary key is composed of both a partition key and a clustering key.
- In our case, the partition key is the column groupid, and the clustering key is the column username. You probably have mentioned the STATIC in the group_name column definition. The STATIC clause is used for special single value columns that are shared by all partition rows. A static column is also called a descriptive column of the partition key.
- Static columns cannot be used as a primary key. When using a SELECT statement, the same applies for groups as it did for the user's table. A SELECT on the groups table would yield no result, since we have not yet inserted any data.

Table properties and options

```
DESCRIBE groups;  
CREATE TABLE intro_cassandra.groups (  
  groupid int,  
  username text,  
  age int,  
  group_name text static,  
  PRIMARY KEY (groupid, username)  
) WITH CLUSTERING ORDER BY (username ASC)  
  AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}  
  AND compaction = {'class':  
    'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy',  
    'max_threshold': '32', 'min_threshold': '4'}  
  AND default_time_to_live = 0  
  AND gc_grace_seconds = 864000  
  AND memtable_flush_period_in_ms = 0;
```

- If we DESCRIBE the table, then we can see the table's definition and properties. Let's explain some of the new things you can see on this screen. First, by default, data inside the partition is ordered by the clustering key username in ascending order. The WITH CLUSTERING ORDER BY parameter can be used to order the data inside the partition. The default_time_to_live means that you can set an expiration time for the data in the table. For example, an offer that is valid only for 5 minutes. After 5 minutes, the data is deleted.
- Time-To-Live, or TTL, can also be set at the operation level. For example, with an INSERT, meaning that only that row will be expired after the TTL. Write data, such as inserts, updates, or deletes is flushed from memory on disk in three situations. When the memtable is full, when the commit log is full, or at a specific interval. This setting refers to the per table setting, specific interval of flushing data to disk. By default, this is 0, meaning not activated. What you can see in this slide are only a few of the Cassandra table properties and options.

ALTER TABLE

- Add new columns
- Drop existing columns
- Rename columns
 - Regular columns
 - Clustering keys
- Changes table properties
- Restriction: Altering PRIMARY KEY columns is not supported
- Restriction: Changing data type of an existing column is not supported

```
ALTER TABLE groups ADD occupation TEXT;
```

```
ALTER TABLE groups DROP age;
```

```
ALTER TABLE groups RENAME username to  
user_name;
```

```
ALTER TABLE groups WITH  
default_time_to_live=10;
```

- As in the case with keyspaces, tables can also be altered. You can add new columns to the table schema, drop columns from the table schema, rename columns, but be aware that it works for regular columns and clustering keys, but is not supported for partition keys, and change table properties.
- Please note that while we are mentioning adding and deleting columns, these operations can be applied only to regular columns, not to primary key columns. In Cassandra, the primary key, once defined, cannot be changed, since it determines the way the data is stored at cluster and node level. If you do want to change it, you will need to create a new table and reimport the data. Changing the data type for an existing column is not supported. You can delete a table using either the TRUNCATE or DROP commands.

TRUNCATE and DROP TABLE commands

- Truncate: Removes all data from a table but not the table's schema definition
- Truncate: Needs Consistency ALL and all replicas available
- Drop: Removes all the data, including schema
- Cassandra takes a snapshot before truncating or dropping the table

```
TRUNCATE TABLE groups;  
TRUNCATE groups;  
DROP TABLE groups;
```

- Truncate removes all the data from the specified table, but not the table's defined schema. One thing to be aware of with the truncate command is that it requires an all consistency. Thus, all replicas need to be available. The drop command removes all data and the tables defined schema. Before truncating or dropping the data, Cassandra takes a snapshot of the data as a backup.

Recap

In this video, you learned that:

- Data in Cassandra is organized logically in tables
- A table's metadata specifies the primary key instructing Cassandra how to distribute the table data at cluster and node level
- You can add Time-To-Live (TTL) at table level meaning that you can expire (delete) all data that has surpassed the TTL
- You can modify the columns and column names, but only for regular columns
- A primary key, once defined at table creation, cannot be modified
- You can either drop a table or truncate its data